Versión 1.4

Paradigmas de Programación

Paradigma Orientado a Eventos

ING. DAVID CECCHI
ING. JACOBO DOMINGUEZ
ING. NICOLÁS TIZZANO

UNIVERSIDAD TECNOLOGICA NACIONAL
Facultad Regional San Nicolás
Ingeniería en Sistemas

Paradigma Orientado a Eventos

En contraposición a la programación secuencial clásica (estructurada) donde las instrucciones se ejecutan una tras otra y las intervenciones del usuario solo ocurren en aquellos puntos donde el programador lo haya determinado, la programación orientada a eventos permite al usuario interactuar con la aplicación en cualquier momento de la ejecución.
La aplicación se encuentra "suspendida" la mayor parte del tiempo, reaccionando solo cuando algo externo ocurre, es decir, el flujo de ejecución no lo dicta el programa sino el entorno. Dicho de otra manera, la aplicación no avanza por su propia voluntad; avanza cuando un elemento externo la convoca.
Los entornos de desarrollo que permiten implementar este paradigma, suelen proporcionar un conjunto de componentes prediseñados para ser utilizados/ubicados dentro de otros componentes que funcionan como contenedores (formularios, ventanas, páginas, etc.) de los mismos.
Estos componentes prediseñados pueden entenderse como objetos: tienen una serie de propiedades y un conjunto de funciones o métodos; adicionalmente cuenta con la capacidad de responder frente a determinados estímulos (sucesos/eventos).
Sintetizando, la programación orientada a eventos es un paradigma de programación en el que tanto la estructura como la ejecución de los programas van determinados por sucesos que ocurren en el sistema.
Un evento es un estímulo que la aplicación reconoce (puede detectar) y sobre el cual tiene la facultad de responder. La forma en que responda frente a un determinado evento, dependerá del código que incorporó el programador para dicho evento.
Los eventos generalmente son provocados por el usuario al interactuar con la interfaz de la aplicación (ej.: un click sobre un botón, seleccionar una fila de la grilla, cerrar una ventana, arrastrar un componente, etc.), pero también pueden ser desencadenados internamente por la propia aplicación (ej.: cada pulso de un temporizador, arribo de datos desde la red, etc.).
Una guía útil para comenzar a codificar bajo éste paradigma consiste en dar respuesta a las siguientes preguntas:

- ¿**Qué** se debe hacer? Para determinar la tarea a realizar.
- ¿**Quién** lo debe hacer? Para identificar el componente que la debe realizar.
- ¿**Cuándo** se debe hacer? Para identificar el evento, del componente hallado en el punto anterior, en el cual se debe codificar.

Nota: la introducción al paradigma y sus características fueron tratadas en la Unidad 1.

Formularios

Un formulario representa una ventana, opera como un contenedor de controles mediante los cuales el usuario interactúa con la aplicación. A su vez, son objetos por lo tanto cuentan con sus propias propiedades, métodos y eventos.
En general, la mayoría de los proyectos que se desarrollan requieren de múltiples formularios.
Algunas de las propiedades, métodos y eventos más importantes vinculados a los formularios son:

Propiedad	**Descripción**
AcceptButton	Determina el botón que será invocado al presionar la tecla ENTER.
CancelButton	Determina el botón que será invocado al presionar la tecla ESC.
ControlBox	Determina si el formulario debe presentar el conjunto de iconos Minimizar/Maximizar/Cerrar
FormBorderStyle	Determina la apariencia del borde del formulario.
ShowInTaskBar	Determina si el formulario debe presentarse en la barra de tareas.
StartPosition	Determina la posición del formulario al presentarse por primera vez.
Text	Determina el texto presentado como titulo del formulario.
Método	**Descripción**
Show	Permite mostrar el objeto.

ShowDialog	Permite mostrar el objeto en forma modal.
CenterToScreen	Centra el objeto en pantalla.
Close	Cierra el objeto.
Dispose	Libera los recursos utilizados por el objeto.
Evento	**Se invoca cuando:**
Load	Se carga el objeto en memoria.
Activate	Se activa el objeto.
FormClosed	Se cierra el objeto.

Propiedades, Métodos y Eventos Básicos

Propiedad	**Descripción**
Backcolor	Determina el color de fondo.
Enabled	Determina si el objeto está habilitado, y por lo tanto si puede responder a eventos.
Font	Determina la fuente a utilizar por el objeto.
ForeColor	Determina el color del primer plano.
Image	Determina la imagen a utilizar.
TabIndex	Determina el orden de tabulación.
Text	Determina el texto presentado.
TextAlign	Determina la alineación del texto presentado.
Visible	Determina si el objeto se encuentra visible.
Método	**Descripción**
Focus	Establece el foco en el objeto.
Evento	**Se invoca cuando:**
Click	Se pulsa el botón primario del Mouse sobre el objeto.
DoubleClick	Se pulsa 2 veces el botón primario del Mouse sobre el objeto.
Enter	El objeto recibe el foco.
KeyDown	Se pulsa una tecla mientras el objeto tiene el foco.
KeyPress	Se pulsa y suelta una tecla mientras el objeto tiene el foco.
KeyUp	Se suelta una tecla mientras el objeto tiene el foco.
Leave	El objeto perdió el foco.
MouseDown	Se pulsó un botón del mouse sobre el objeto.
MouseMove	Se movió el cursor del mouse sobre el objeto.
MouseUp	Se soltó un botón del mouse sobre el objeto.

Nota: Se debe tener en cuenta que no todos los objetos cuentan con las mismas propiedades y métodos ni responden a los mismos eventos.

Objetos Visuales de Uso Frecuente

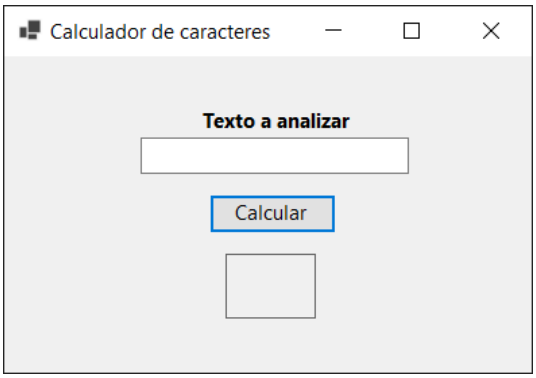
Objeto	**Descripción**	
Button	Los botones se utilizan para iniciar, interrumpir o detener procesos.	
	Evento	**Descripción**
	Click	Invocado cuando el usuario realiza un Click sobre el objeto
CheckBox	Las cajas de verificación se utilizan para presentar diversas opciones en un formulario, de las cuales el usuario podrá seleccionar una o varias de ellas.	
	Propiedad	**Descripción**
	Checked	Determina si la casilla de activación se encuentra activa.
	Evento	**Descripción**
	CheckedChanged	Invocado cuando el usuario modifica el estado de la casilla de verificación

<table>
<tr><td>ComboBox</td><td colspan="2">Las cajas combinadas consisten en una lista desplegable de opciones. Se diferencian principalmente de las cajas de listado en que las cajas combinadas le permiten al usuario introducir nuevos valores pero resignando la posibilidad de realizar selecciones múltiples.</td></tr>
<tr><td></td><td>Propiedad</td><td>Descripción</td></tr>
<tr><td></td><td>DataSource</td><td>Determina el origen de datos del objeto.</td></tr>
<tr><td></td><td>DisplayMember</td><td>Determina el nombre del campo a presentar de los items.</td></tr>
<tr><td></td><td>DropDownStyle</td><td>Determina el estilo del objeto.</td></tr>
<tr><td></td><td>Items</td><td>Hace referencia a la colección de ítems del objeto.</td></tr>
<tr><td></td><td>SelectedIndex</td><td>Determina el índice del ítem seleccionado.</td></tr>
<tr><td></td><td>SelectedItem</td><td>Hace referencia al ítem seleccionado.</td></tr>
<tr><td></td><td>SelectedValue</td><td>Determina el valor real asociado al ítem seleccionado.</td></tr>
<tr><td></td><td>Text</td><td>Hace referencia al texto correspondiente al ítem seleccionado.</td></tr>
<tr><td></td><td>ValueMember</td><td>Determina el nombre del campo que se utilizara como valor real de los ítems.</td></tr>
<tr><td></td><td>Evento</td><td>Descripción</td></tr>
<tr><td></td><td>SelectedIndexChanged</td><td>Invocado cuando un nuevo ítem es seleccionado.</td></tr>
<tr><td>DateTimePicker</td><td colspan="2">Representan una lista desplegable que permite al usuario seleccionar una Fecha y Hora.</td></tr>
<tr><td></td><td>Propiedad</td><td>Descripción</td></tr>
<tr><td></td><td>Format</td><td>Determina el formato de la fecha y hora presentado por el objeto.</td></tr>
<tr><td></td><td>MaxDate</td><td>Determina la fecha y hora máximas que pueden ser seleccionadas.</td></tr>
<tr><td></td><td>MinDate</td><td>Determina la fecha y hora mínimas que pueden ser seleccionadas.</td></tr>
<tr><td></td><td>Value</td><td>Determina la fecha y hora especificadas.</td></tr>
<tr><td></td><td>Evento</td><td>Descripción</td></tr>
<tr><td></td><td>ValueChanged</td><td>Invocado cuando la fecha es modificada.</td></tr>
<tr><td>Label</td><td colspan="2">Las etiquetas son utilizadas para mostrar mensajes, títulos o presentar resultados. Se caracterizan porque no permiten que el usuario ingrese texto en ellas en forma directa.</td></tr>
<tr><td>MaskedTextBox</td><td colspan="2">Objetos que restringen el ingreso de datos al usuario, obligándolo a cumplir con el formato de la máscara previamente especificada.</td></tr>
<tr><td></td><td>Propiedad</td><td>Descripción</td></tr>
<tr><td></td><td>Mask</td><td>Determina la máscara de entrada del objeto.</td></tr>
<tr><td></td><td>MaskCompleted</td><td>Determina si el usuario ha completado la máscara de entrada.</td></tr>
<tr><td>MenuStrip</td><td colspan="2">Objetos utilizados para permitir al usuario la selección, en forma visual, de una operación a realizar.</td></tr>
<tr><td>PictureBox</td><td colspan="2">Objetos utilizados para presentar una imagen.</td></tr>
<tr><td>RadioButton</td><td colspan="2">Los botones de opción se utilizan para agrupar un conjunto de opciones, de las cuales sólo se puede elegir una de ellas.
Para crear varios grupos de botones de opción, a cada grupo se lo debe colocar dentro de un contenedor diferente.</td></tr>
<tr><td></td><td>Propiedad</td><td>Descripción</td></tr>
<tr><td></td><td>Checked</td><td>Determina si la casilla de activación se encuentra activa.</td></tr>
<tr><td></td><td>Evento</td><td>Descripción</td></tr>
<tr><td></td><td>CheckedChanged</td><td>Invocado cuando el usuario modifica el estado de la casilla de verificación</td></tr>
<tr><td>TextBox</td><td colspan="2">Las cajas de texto se utilizan para la entrada de datos.</td></tr>
<tr><td></td><td>Propiedad</td><td>Descripción</td></tr>
<tr><td></td><td>MaxLength</td><td>Determina la cantidad máxima de caracteres que el usuario podrá tipear en el objeto.</td></tr>
<tr><td></td><td>PasswordChar</td><td>Determina el carácter que reemplazará a aquellos tipeados.</td></tr>
</table>

Proyecto Introductorio

Se presenta un mini proyecto constituido por un único formulario.
El objetivo de la aplicación es permitir al usuario conocer la cantidad de caracteres de un texto ingresado previamente.
Para ello, se procede a:

1. Diseñar la interfaz.



2. Codificar eventos.

```
private void frmCalculador_Load(object sender, EventArgs e)
{
   //una vez cargado (Load) el formulario en memoria, centrarlo en pantalla

   this.CenterToScreen();
}

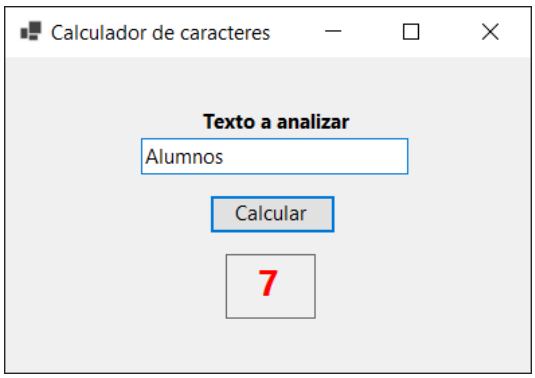
private void btnCalcular_Click(object sender, EventArgs e)
{
   //al clickear sobre el botón Calcular:

   //declaracion de variable
   int cant;

   //se determina la cantidad de caracteres ingresados en la caja de texto
   cant = txtTextoAnalizar.Text.Length;

   if (cant == 0)
   {
      //si la cantidad es cero, la etiqueta se presenta vacía"
      lblCantCaracteres.Text = string.Empty;
   }
   else
   {
      //si la cantidad no es cero, en la etiqueta se presenta la cantidad (int) calculada
      //pero convertida a cadena (string)
      lblCantCaracteres.Text = cant.ToString();
   }
}
```

3. Testear.



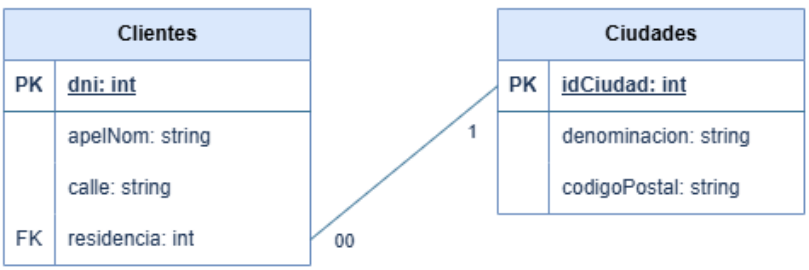
SQL

Con el objetivo de abordar el tema Persistencia en Bases de Datos en el contexto de aplicaciones desarrolladas implementando POE, se realizará una breve introducción al lenguaje SQL.
Una base de datos es una colección de datos relacionados, almacenados de forma persistente.
Las bases de datos se encuentran gestionadas por un Sistema Gestor de Bases de Datos (SGBD), como por ejemplo SQL Server, Oracle, PostgreSQL, MySQL, entre otros.
El objetivo principal de un SGBD es proporcionar un medio eficiente para la construcción, manipulación y protección de bases de datos (frente a accesos no autorizados y funcionamiento defectuoso del hardware o software que desencadenan caídas de servicio), asegurando la integridad y fiabilidad de los datos almacenados, entre otras tareas.
El lenguaje de consulta estructurado, SQL (Structured Query Language), es un lenguaje declarativo que permite la creación y mantenimiento de una base de datos relacional a través de instrucciones, operando tanto sobre su estructura como en la gestión de los datos.
Una base de datos relacional recurre a un grupo de tablas relacionadas entre sí para almacenar datos y representar las vinculaciones entre ellos. Cada tabla está compuesta por filas (registros) y columnas (campos). Para cada tabla se debe definir una clave principal.
La clave principal podrá estar constituida por uno o más campos que, en conjunto, permitan identificar unívocamente a cada registro almacenado. En otras palabras, no pueden existir dos registros con idénticos valores en los campos que constituyen la clave principal.
A continuación, a modo de ejemplo:

Diagrama de Clases



Diagrama Relacional



Datos Almacenados en Base de Datos

DNI	ApelNom	Calle	Residencia
25014778	Cejas, Lujan	Maipu 66	1
33450779	Martinez, Juan	Savio 1022	3
45005123	Durand, Andrea	Bolivar 18	1
28099200	Gomez, Fernando	Balcarce 44	2

IdCiudad	Denominacion	CodigoPostal
1	San Nicolás	2900
2	Ramallo	2915
3	Pergamino	2700
4	Rosario	2000

A través de SQL es posible, entre otras operaciones: insertar, actualizar, eliminar y solicitar registros.

INSERTAR

Sintaxis Básica:
INSERT INTO NombreDeTabla **(**NombreCampo1, …**) VALUES (**ValorCampo1, ….**)**
Nota: los valores para campos de texto y fechas deben encerrarse entre comillas simples.

Ejemplo:
INSERT INTO Clientes (DNI, ApelNom, Calle, Residencia) VALUES (20123789, 'Romero', 'Bolivar 23', 3)

ACTUALIZAR

Sintaxis Básica:
UPDATE NombreDeTabla **SET** NombreCampoActualizar = ValorNuevo, … **WHERE** Condición
Nota: en caso de no definir una condición se actualizarán todos los registros de la tabla.

Ejemplo:
UPDATE Clientes SET Calle = 'Maipu 34' WHERE DNI = 20123789

ELIMINAR

Sintaxis Básica:
DELETE NombreDeTabla **WHERE** Condición
Nota: en caso de no definir una condición se eliminarán todos los registros de la tabla.

Ejemplo:
DELETE Clientes WHERE NDoc = 20123789

SOLICITAR

Sintaxis Básica:
SELECT NombreCampo1, ….. **FROM** NombreTabla **WHERE** Condición
Nota: en caso de no definir una condición se retornarán todos los registros de la tabla.

Ejemplo:
SELECT DNI, ApelNom FROM Clientes WHERE Residencia = 1

Asincronismo

Conceptos Básicos

Para la explicación se plantea un paralelo entre un software y una obra de construcción.

- **Aplicación**: es el software en su completitud (instrucciones, componentes, dependencias, etc.). Se puede interpretar como la obra completa con todas las indicaciones necesarias para llegar a buen puerto.
- **Proceso**: el lanzamiento de una aplicación lleva al sistema operativo a crear una serie de procesos necesarios para lograr su ejecución, asignándole a cada uno de ellos su propio espacio de memoria y

todos los recursos que requiera. La cantidad de procesos creados, al intentar ejecutar una aplicación, tiene su origen en tres motivos bien diferenciados:

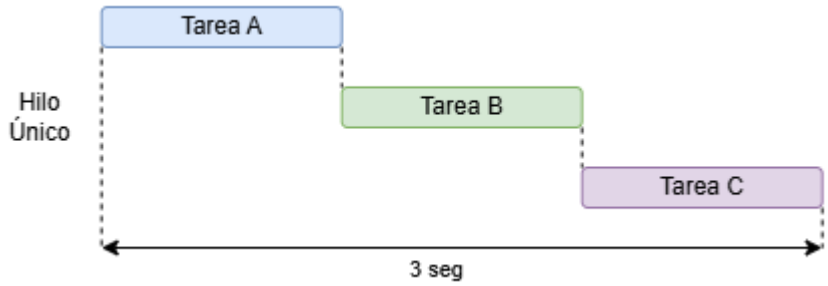
1. **Lanzamiento inicial**: al ejecutar la aplicación (doble clic o por comando), se le solicita al sistema operativo la creación del proceso base que la representa.
2. **Código explícito insertado por el desarrollador**: según la arquitectura definida para la aplicación, esta puede solicitar por código la creación de procesos adicionales (ej. pestañas de Chrome).
3. **Reglas fijas según el tipo de ejecutable**: para algunos tipos de aplicaciones se encuentra predefinida una lista de procesos que el sistema operativo debe invocar para que una aplicación de ese tipo pueda correr, ej.: en Windows, toda aplicación de consola recibe adicionalmente, por parte del sistema operativo, un proceso conhost.exe que gestiona su ventana.

Cada proceso puede interpretarse como una etapa de la obra (ej. armar cimientos, levantar paredes, construir techo, instalar aberturas, cableado, etc.).
Cuando la obra se pone en marcha (doble clic sobre la aplicación), el capataz (sistema operativo) abre las etapas (procesos) que correspondan: la etapa inicial por el propio arranque de la obra, las etapas adicionales que la obra indique explícitamente por decisión del arquitecto (desarrollador); y las etapas que exigen las normas preestablecidas para construcciones de ese tipo, asignándole a cada etapa los recursos materiales que requiera (memoria, hilos, etc.).
Un proceso no puede leer ni escribir en la memoria de otro proceso de manera directa (salvo casos excepcionales), esta restricción garantiza seguridad y estabilidad.

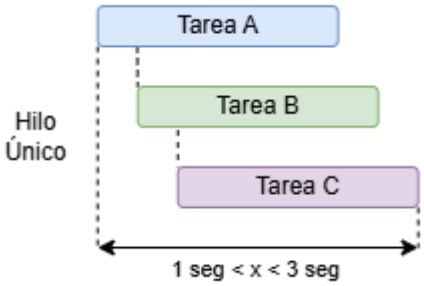
- **Hilo (thread)**: es el ente ejecutor (albañil) de instrucciones, siempre se encuentra asignado a un proceso determinado, y no tiene permitido cambiar de proceso.
  Un hilo ejecuta las instrucciones que constituyen al proceso que pertenece y lo hace ejecutando instrucción por instrucción, siguiendo el orden allí definido: entra a una subrutina, ejecuta sus líneas una a una, sale, entra a la siguiente, ejecuta sus líneas una a una, sale, etc.
  Sin un hilo, el proceso es solo una lista de instrucciones residente en memoria que nadie ejecuta. Por ello, todo proceso nace con al menos un hilo garantizado: el hilo principal. Si la obra se ha planificado con una determinada arquitectura, podría ser factible contar con varios albañiles (hilos) trabajando en simultáneo sobre una misma etapa (proceso).
  Cabe aclarar que, los hilos no se refieren a hilos propios/físicos del procesador, sino que refieren a hilos del sistema operativo: una abstracción de software creada, programada y eliminada por el sistema operativo, que luego reparte entre las unidades de ejecución físicas disponibles.

Modelos de Procesamiento

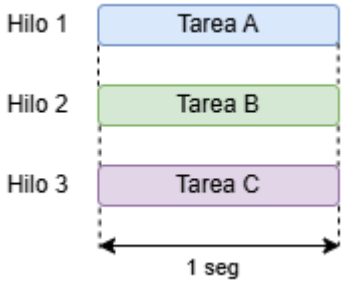
- **En serie**: es la ejecución estrictamente lineal (secuencial) de tareas, donde cada una de ellas debe finalizar por completo antes de que la siguiente pueda comenzar.



- **Concurrencia**: es la capacidad de gestionar múltiples tareas en progreso, intercalando su ejecución, sin llegar a ejecutarlas en el mismo instante. Puede desarrollarse implementando un único hilo, el cual va saltando de una tarea a otra durante los instantes que es liberado.



- **Paralelismo**: es la capacidad de gestionar múltiples tareas en el mismo instante de tiempo, requiriendo obligatoriamente hardware multicore (múltiples núcleos físicos) ya que de lo contrario se estaría implementando un pseudo paralelismo que en los hechos implicaría una concurrencia.



Sincrónico vs Asincrónico

- **Síncrono**: cuando el hilo (albañil) llega a una operación cuya ejecución requiere de un tiempo para completarse, por hacer un uso intensivo de CPU (ej. calculo complejo) o esperar a una respuesta externa (ej. devolución de datos del servidor de base de datos, una operación en disco, un temporizador, etc.), se ve impedido de continuar con las instrucciones posteriores hasta que la misma finalice. En consecuencia, durante ese período, el hilo estará ocupado (ejecutando la operación que representa un cálculo intensivo) o bloqueado (esperando obtener una respuesta de una operación externa). Una vez finalizada la operación que retuvo al hilo, se continuará con la próxima.
- **Asíncrono**: cuando el hilo (albañil) llega a una operación cuya ejecución requiere de un tiempo para completarse, en vez de permanecer bloqueado durante ese período (ej. por espera de una respuesta externa), se libera y queda disponible. Si la operación en cuestión es un cálculo intensivo de CPU, la liberación del hilo invocante solo es posible si el cálculo se delega explícitamente a otro hilo (ej. mediante Task.Run); el trabajo en sí no deja de requerir un hilo activo, solo se traslada a uno distinto del que realizó la invocación.
  Cuando aquella operación que requería tiempo ha finalizado, se retoma la ejecución desde el punto en que había sido suspendida, y lo hace a través del mismo hilo u otro disponible.
  No se deben confundir los conceptos *asincronismo* con *paralelismo*: asíncrono implica que un hilo no permanece esperando, sino que se libera para continuar con otra tarea del mismo proceso; mientras que paralelismo implica contar con múltiples hilos (contratar más de un albañil) para que trabajen al mismo tiempo dentro del mismo proceso.

En consecuencia, una subrutina/método se considera asincrónico cuando su ejecución no bloquea al hilo que lo invoca pese a no haber terminado la ejecución de sus instrucciones. El hilo llamador, aquel que invocó a la subrutina/método, recupera el control (normalmente en forma de un Task) y queda libre para ejecutar o invocar otra operación. La finalización de la subrutina/método se comunica más adelante, sin necesidad que el hilo haya tenido que permanecer esperando.
En C# se utiliza el modificador async y la instrucción await para replicar el asincronismo, pero no son obligatorios para lograrlo. Se deben tener presente dos situaciones:

1. Un método puede ser asincrónico y no implementar async/await, retornando un Task generado por una operación interna.

2. Un método con el modificador async pero sin ningún punto de liberación (await) se ejecuta en forma sincrónica.

Async-Await-Task

Para indicar que una subrutina del proceso es asincrónica se utiliza en su firma el modificador **async**. Este tipo de subrutinas generalmente retorna un objeto de la clase **Task** o **Task<T>** (también puede retornar void, pero es una combinación cuyo uso se recomienda limitar a manejadores de eventos).
Al declarar una subrutina con el modificador async, queda habilitada para el uso de la instrucción **await** en su interior, y simultáneamente se le otorga la capacidad de ser transformada en una "*máquina de estados*" (almacenando información sobre las variables locales, el punto de ejecución al cual se llegó y otros datos asociados) de forma tal que al finalizar el Task asociado al await, el hilo que sea asignado cuente con la información necesaria para continuar con la ejecución desde el punto en que había sido liberado el hilo anterior. Es decir, la secuencia lógica del código vive en la máquina de estados de la subrutina, no en el hilo.
Pero se debe tener presente que:

1. El modificador async no convierte por sí mismo una subrutina sincrónica en asincrónica.
2. La instrucción await no genera un nuevo hilo.
3. Un Task representa una tarea (operación); no representa un hilo.

La instrucción await suspende la ejecución de la subrutina que la contiene hasta tanto la tarea representada por el Task asociado finalice; esta pausa la realiza sin mantener el hilo bloqueado, es decir, liberándolo para que pueda ser utilizado por el mismo proceso en la ejecución de otra subrutina.
Por lo tanto, async, await y Task normalmente operan en conjunto.
Todo Task cuenta con un estado asociado que puede tomar los valores *"pendiente", "completado con éxito", "completado con una excepción" y "cancelado"*, entre otros estados posibles. Los Task, al finalizar, pueden no devolver un resultado (similar a void en métodos sincrónicos) o pueden retornar un tipo de dato determinado (ej. Task<int>).
La operación invocada (ej. conexión.OpenAsync()) es quien genera el Task que representa el trabajo en curso. La instrucción await toma ese Task y libera el hilo de ejecución mientras espera a que dicho Task cambie de estado. Cuando el Task finaliza (estado "completado con éxito", "completado con una excepción" o "cancelado"), retorna, si corresponde, el valor obtenido, y la ejecución del proceso se retoma (gracias a la "máquina de estados") desde la instrucción que seguía al await. La ejecución la retoma alguno de los hilos disponibles del proceso, pudiendo ser el mismo u otro distinto al liberado por el await.

GetAwaiter().GetResult()

La instrucción bloquea el hilo hasta que el Task termina y luego devuelve el resultado o propaga la excepción. Es una forma síncrona de consumir una operación asincrónica representada por un Task, generalmente debe evitarse en aquellas situaciones que sea posible utilizar await.

Invocando Operaciones Asincrónicas

Para ejemplificar distintas alternativas al momento de invocar un método asincrónico se recurre a un objeto "conn", instancia de la clase SqlConnection, al cual se le invoca el método OpenAsync(), encargado de establecer la conexión a una base de datos:

- **conn.OpenAsync()**: el hilo seguirá con la siguiente instrucción sin esperar la respuesta del método, por lo cual, si la próxima instrucción requiere de los resultados de *OpenAsync()* y los mismos aún no se encuentran disponibles, se generará una excepción.
- **await conn.OpenAsync()**: el hilo será liberado; el flujo de ejecución se retomará (a través de algún hilo disponible, el mismo u otro distinto) cuando la operación finalice.
- **conn.OpenAsync().GetAwaiter().GetResult()**: el hilo permanecerá bloqueado hasta que *OpenAsync()* finalice, lo cual implica que el flujo de ejecución se llevará de manera sincrónica (bloqueante).
- **Task tarea = conn.OpenAsync();…….;await tarea**: en la línea *Task tarea = conn.OpenAsync()* el hilo seguirá con la siguiente instrucción sin esperar la respuesta del método. Al llegar a *await tarea*, si la

tarea aún no finalizó, el hilo será liberado y el flujo de ejecución se retomará (a través de algún hilo disponible, el mismo u otro distinto) cuando la operación finalice. Si, por el contrario, al llegar a *await tarea* la tarea ya finalizó, el flujo de ejecución continuará de inmediato, sin incurrir en la liberación del hilo.

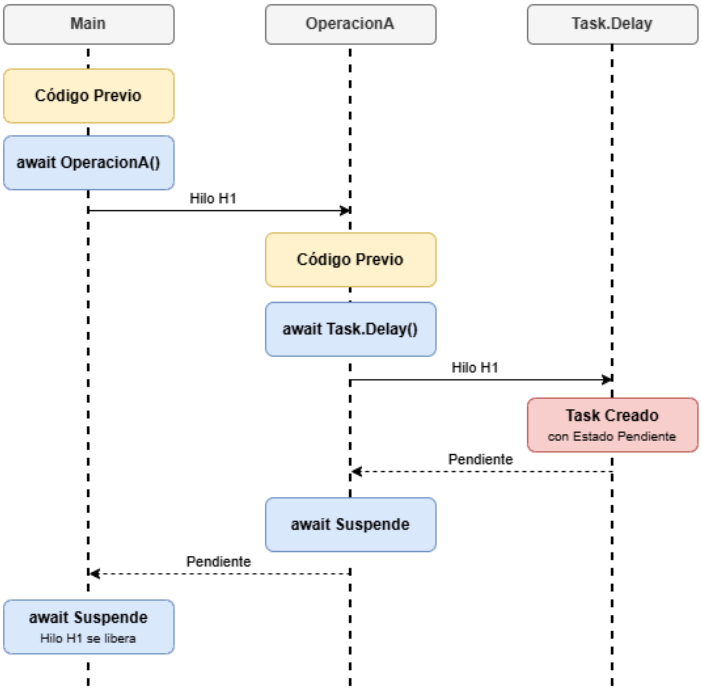
Diagramas de Secuencia Simplificados

- **Código fuente:**

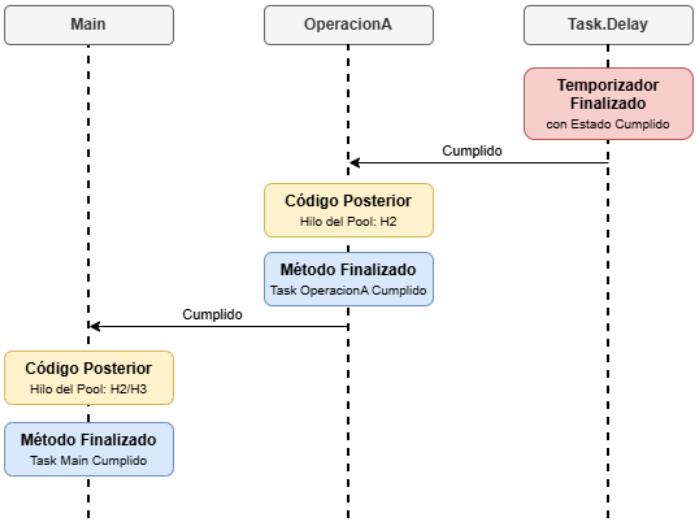
```
static async Task Main(string[] args)
{
    //código previo
    await OperacionA();
    //código posterior
}
```

```
static async Task OperacionA()
{
    //código previo
    await Task.Delay(1000);
    //código posterior
}
```

- **Ida:**



- **Vuelta:**



Using

La instrucción le indica al compilador que, en el momento en que se produzca la salida del bloque (cualquiera sea la razón, incluso por una excepción), se llame automáticamente al método de limpieza del objeto asociado. Para objetos que implementan la interfaz IDisposable, se utiliza en su declaración la instrucción using y el método de limpieza correspondiente se denomina Dispose(), mientras que para aquellos objetos que implementan la interfaz IAsyncDisposable, se utiliza await using y el método de limpieza se denomina DisposeAsync().
Los métodos de limpieza Dispose() y DisposeAsync() no se invocan automáticamente cuando el bloque, que contiene a los objetos, finaliza. Para garantizar su invocación debe utilizarse using/await using o hacerlo explícitamente.
Para visualizar la implicancia del using, si el albañil toma prestada una herramienta (recurso externo) y al finalizar con su tarea, en vez de devolverla al sitio correspondiente (efecto logrado al utilizar using) la deja en cualquier lugar, la herramienta estará ocupando un espacio que no es el adecuado ("molestando") o permaneciendo perdida hasta que finalice la obra y alguien la encuentre y ubique donde corresponda. Ninguna de las situaciones, resuelven el dilema de manera eficiente, ni tan rápido ni confiable como sucedería si el mismo trabajador la devolviera apenas termina su tarea.
Ejemplificando, si en un bloque se declara un objeto para conexión a base de datos, al finalizar dicho bloque y cuando el garbage collector lo considere adecuado (podría no ser inmediatamente finalizado el mismo), se procederá a liberar la memoria asignada al objeto, pero los recursos administrados por el objeto en cuestión podrían permanecer ocupando recursos propios (memoria no administrada, sockets de red, referencias/handles a archivos, sesiones abiertas en un servidor remoto, etc.) salvo que: se haya recurrido a la instrucción using al momento de su declaración o invocado explícitamente al método de limpieza correspondiente antes de salir.
Si los recursos administrados por el objeto no son liberados explícitamente, la conexión podría quedar a la espera de ser recolectada por el garbage collector e inaccesible para la aplicación, mientras que los recursos que ella administraba quedarían "huérfanos", permaneciendo vigentes durante un tiempo mayor al deseado, generando consumos innecesarios y mermas de rendimiento.

Inyección de Dependencias

La Inyección de Dependencias (DI) es un patrón de diseño mediante el cual una clase recibe las instancias de las clases que necesita para funcionar (sus "dependencias") desde afuera (generalmente a través de su constructor, aunque también puede hacerlo mediante sus propiedades o métodos), en lugar de crearlas ella misma. La inyección por constructor es la forma más frecuente al garantizar que el objeto nunca exista sin sus dependencias.
Si al aplicar el patrón se opta por referenciar a interfaces y no clases de manera directa, se ganará flexibilidad. Dicha flexibilidad se reflejará en la posibilidad de intercambiar implementaciones sin modificar el código cliente. Beneficios que otorga:

- **Bajo acoplamiento**: las clases dependen de abstracciones (interfaces), no de implementaciones concretas.
- **Testeabilidad**: permite inyectar una clase de prueba (por ejemplo, que trabaje con datos en memoria, archivos, etc.) para los tests unitarios, sin necesidad de una base de datos real.
- **Mantenibilidad**: cambiar una implementación (por ejemplo, pasar de un repositorio que trabaja con SQL Server a uno en memoria u otro que emplee un gestor de base de datos diferente) no requiere modificar el código de las clases que la consumen.
- **Gestión centralizada del ciclo de vida de los servicios**: el contenedor de dependencias decide cuando debe crearse, compartirse y destruirse cada servicio Transient, Scoped o Singleton. Es posible aplicar el patrón de DI sin utilizar un contenedor de dependencias (IServiceProvider); en los

ejemplos dados en práctica el contenedor de dependencias es el objeto *provider*, aquel retornado por *services.BuildServiceProvider()*.
En el contenedor de dependencias, básicamente se deben registrar:

- Las interfaces utilizadas en los constructores de clases, para lograr el desacoplamiento.
- Las clases concretas (ej.: frmPersona) que hacen uso de dependencias para que el contenedor pueda resolver su árbol de dependencias.
  Si una clase con alguna dependencia en su constructor no es registrada en el contenedor de dependencias, aún si la interface de la cual depende sí lo está, dicha clase no podrá ser instanciada a través del contenedor.

Adicionalmente, se puede optar por incluir aquellas clases concretas que, si bien en el presente no cuentan con dependencias, se ha decidido que el contenedor se encargue de su instanciación: porque en un futuro posiblemente se le determine una dependencia o para mantener una arquitectura consistente (prolija/uniforme).

Ejemplo sin contenedor

```
IPersonaRepository personaRepo = new PersonaRepositorySQL();
IPersonaService personaService = new PersonaService(personaRepo);
ApplicationConfiguration.Initialize();
Application.Run (new frmPersona(personaService));
```

Ejemplo con contenedor

```
var services = new ServiceCollection();
services.AddScoped<IPersonaRepository, PersonaRepositorySQL>();
services.AddScoped<IPersonaService, PersonaService>();
services.AddTransient< frmPersona>();
using var provider = services.BuildServiceProvider();
var mainForm = provider.GetRequiredService<frmPersona>();
ApplicationConfiguration.Initialize();
Application.Run(mainForm);
```

Tipos de Servicios

- **Transient**: si para dar respuesta a un pedido (request) se requiere inyectarlo varias veces, en cada una de ellas se inyecta una nueva instancia. En window forms, cada vez que se pide (inyecta) un formulario se genera una nueva instancia. Útil para servicios livianos y cuando no es necesario compartir el estado dentro de un mismo pedido.
- **Scoped**: si para dar respuesta a un pedido (request) se requiere inyectarlo varias veces, en cada una de ellas se inyecta la misma instancia. Útil para repositorios, servicios y unitwork.
- **Singleton**: una única instancia para toda la vida de la aplicación, es decir, se utiliza la misma instancia para todos los pedidos (inyecciones) de todos los usuarios. Útil para servicios inmutables, configuraciones de solo lectura, etc.

Guía Práctica

A través del siguiente diagrama se brinda un resumen asociado a la ejercitación dada en clase donde se combinan todos los conceptos adquiridos hasta la presente unidad.

```
public class Persona
{
    public int IdPersona { get; set; }
    public int NDoc { get; set; }
    public string Apellido { get; set; }
    .
    .
}
```

1 Creación de la Clase Entidad

Crear la clase (Persona) a partir del origen de datos involucrado, en este caso una base de datos SQLServer.
La clase deberá respetar la estructura de la tabla (Personas) sobre la cual interesa operar, es decir, cada campo de la tabla se representará como una propiedad encapsulada (getters y setters).
La etapa de Análisis permitirá elaborar el *modelo de datos conceptual*. A continuación, la etapa de Diseño tomará este modelo como punto de partida y desarrollará el *modelo de datos relacional*, que será utilizado durante la etapa de Implementación para construir la base de datos en el SGBD seleccionado.
Paradigmas implicados: POO.

2 Creación de la Interface

Definir el contrato abstracto (IPersonaRepository) que declara toda la funcionalidad que deberían brindar los repositorios interesados en manipular un origen de datos.
Paradigmas implicados: POO.

```
public interface IPersonaRepository
{
    Task<List<Persona>> GetAllAsync();
    Task<int> CreateAsync(Persona persona);
    Task<bool> UpdateAsync(Persona persona);
    Task<bool> DeleteAsync(int id);
    .
    .
}
```

```
public class PersonaRepositorySQL : IPersonaRepository
{
    public async Task<int> CreateAsync(Persona perso)
    {
    .
    .
    string sql = @"INSERT INTO Personas (...) Values (...)";

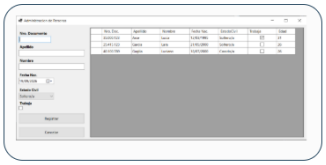
    await using SqlConnection conn = new SqlConnection(CONN_STRING);
    await conn.OpenAsync();
    .
    .
    }
}
```

3 Creación del Repositorio (DAL)

Construir la clase DAL (PersonaRepositorySQL) que implementa la interface (IPersonaRepository), dando cuerpo a la funcionalidad que la interface obliga a brindar.
En este ejemplo puntual, cada operación del repositorio PersonaRepositorySQL recurre a instrucciones SQL para interactuar con una base de datos SQLServer.
Paradigmas implicados: POO y Declarativo-Descriptivo (SQL)

4 Diseño de la Interface de Usuario (UI)

Diseñar la UI a través de la cual el usuario podrá interactuar con los datos.
En este caso, al tratarse de una aplicación de escritorio se realizará a través de un WindowForm, en consecuencia se declararan controles (objetos visuales) y manejadores de eventos para responder a las interacciones del usuario.
Paradigmas implicados: POE y POO.



```
internal static class Program
{
    static void Main()
    {
    .
    .
    services.AddScoped<IPersonaRepository, PersonaRepositorySQL>();
    .
    .
    }
}
```

5 Config. del Contenedor de Dependencias

Registrar en el contenedor de dependencias los servicios necesarios para proporcionar Inyección de Dependencias (DI). A través de ellos se establece la clase que será inyectada (PersonaRepositorySQL) cuando se referencie a una determinada interface (IPersonaRepository).
Conceptos implicados: DI.

6 Inyección del Repositorio

Al solicitar la construcción de una instancia del WindowForm, el contenedor de dependencias resolverá/inyectará en él, a través de su constructor, el repositorio que corresponda según los servicios que le han sido definidos al momento de configurarlo.
Conceptos implicados: DI.

```
public partial class frmPersona : Form
{
    private readonly IPersonaRepository _repo;

    public frmPersona(IPersonaRepository repo)
    {
        InitializeComponent();
        this._repo = repo;
    }
    .
    .
}
```

```
public partial class frmPersona : Form
{
    .
    .
    private async void btnRegistrar_Click(object sender, EventArgs e)
    {
        .
        .
        await _repo.CreateAsync(persona);
        .
        .
    }
}
```

7 Interacción del Usuario

El usuario, al interactuar con el WindowForm (UI), activará eventos que invocarán métodos sobre la instancia del repositorio inyectado. Aquellos métodos actuarán sobre el origen de datos (base de datos, archivo, etc.) que manipula el repositorio.
Conceptos implicados: se combinan todos los anteriores.

Tal como se ha mencionado en el dictado del cursado, al no haber tratado los conceptos 'capa de servicio' (encargada de manejar la lógica de negocios) ni 'modelo de dominio rico' (entidades con validaciones incluidas en su propia definición, inclinándose en su lugar por implementar un 'modelo anémico'), se ha optado inicialmente por incorporar las validaciones dentro del repositorio. Posteriormente, al plantear un segundo repositorio que opera sobre un archivo (csv), se observará el inconveniente con la duplicación de validaciones. De esta manera, se pretende que el alumno visualice la ventaja de trasladarlas a una capa superior (en este caso, la UI) evitando así repetir la misma lógica de validaciones en cada repositorio.
Si bien esta decisión se aparta de las buenas prácticas que aconsejan la separación de responsabilidades entre capas, donde el repositorio debería limitarse exclusivamente al acceso a datos, y la UI limitarse a posibilitar la interacción del usuario: capturar su entrada, presentar datos, comunicar resultados, pudiendo incluir validaciones inmediatas sobre los datos cargados; se ha optado por este camino para evitar aumentar la complejidad de las resoluciones, sin perder el objetivo de la unidad temática.

Contenido

Paradigma Orientado a Eventos ____ *1*
Formularios ____ *1*
Propiedades, Métodos y Eventos Básicos ____ *2*
Objetos Visuales de Uso Frecuente ____ *2*
Proyecto Introductorio ____ *4*
SQL ____ *5*
Asincronismo ____ *6*
Conceptos Básicos ____ 6
Modelos de Procesamiento ____ 7
Sincrónico vs Asincrónico ____ 8
Async-Await-Task ____ 9
GetAwaiter().GetResult() ____ 9
Invocando Operaciones Asincrónicas ____ 9
Diagramas de Secuencia Simplificados ____ 10
Using ____ 11
Inyección de Dependencias ____ *11*
Tipos de Servicios ____ 12
Guía Práctica ____ *13*